INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Ingeniería de Software · Bases de Datos I

Tarea 4

Implementación del modelo físico de la red social estudiantil pascualina en tres SGBD

Informe técnico comparativo: PostgreSQL, MySQL y SQL Server, con datos semiestructurados (JSON) para casos de Big Data

Estudiante: Julian Quintero Copete

Docente: Oralia Cortes

Fecha: 22/10/2026

Contenido

Contenido **2**

1. Introducción **3**

1.1 Contexto y objetivo 3

1.2 Alcance y forma de trabajo 3

2. Justificación de la elección de los tres SGBD **3**

3. Diccionario de datos genérico actualizado **3**

3.1 Tablas y atributos 3

3.2 Relaciones del modelo 5

4. Datos semiestructurados: JSON para Big Data e IoT **6**

4.1 Qué se agregó 6

4.2 Justificación técnica y relación con Big Data e IoT 6

4.3 Tipo usado en cada motor 6

4.4 Consultas de prueba 6

5. Implementación paso a paso **7**

5.1 Cambios comunes a los tres motores 7

5.2 PostgreSQL 17 (pgAdmin 4) 7

5.3 MySQL 8.4 (db<>fiddle) 8

5.4 SQL Server 2022 (db<>fiddle) 9

6. Diccionarios de datos físicos **10**

6.1 Diccionario físico de PostgreSQL 17 10

6.2 Diccionario físico de MySQL 8.4 13

6.3 Diccionario físico de SQL Server 2022 15

7. Dificultades encontradas **18**

7.1 MySQL: CHECK incompatible con ON UPDATE CASCADE (error 3823) 18

7.2 SQL Server: varias rutas de cascada (error 1785) 19

7.3 Tres formas distintas de manejar JSON 19

7.4 Probar sin instalar los motores 19

8. Ventajas y desventajas observadas **19**

8.1 Por motor 19

8.2 Comparación técnica 19

9. Conclusiones generales y propuestas de mejora **20**

9.1 Conclusiones generales del proceso 20

9.2 Propuestas de mejora 20

10. Conclusiones individuales **20**

11. Organización del repositorio Git y video **21**

Referencias **21**

1. Introducción

1.1 Contexto y objetivo

La comunidad pascualina necesita una plataforma informal donde los estudiantes puedan crear perfiles, seguirse entre sí, publicar preguntas y recursos, formar grupos de estudio y programar eventos. En las tareas anteriores se diseñó el modelo entidad-relación, se pasó al modelo lógico con normalización y se implementó el modelo físico en PostgreSQL. En esta tarea el objetivo es **llevar ese mismo modelo a tres sistemas gestores de bases de datos (SGBD)**, documentar cada implementación con un diccionario de datos físico y **agregar un tipo de dato semiestructurado (JSON)** justificado con un caso de uso de Big Data o IoT.

1.2 Alcance y forma de trabajo

El reto está planteado para equipos, pero este trabajo lo desarrollé de forma **individual** (grupo 6). Por eso las secciones de conclusiones grupales se presentan como conclusiones generales del proceso y las propuestas de mejora incluyen lo que haría con apoyo de un equipo.

Los tres scripts se ejecutaron sin errores en su versión final. PostgreSQL se probó de forma local con pgAdmin 4. MySQL y SQL Server se probaron en la herramienta en línea db<>fiddle (MySQL 8.4 y SQL Server 2022 Express sobre Linux), por lo que no se instalaron localmente. Esto tiene una consecuencia práctica: la herramienta ya entrega una base de datos creada, así que para esas pruebas se usó una versión del script sin las instrucciones de crear la base de datos.

2. Justificación de la elección de los tres SGBD

Se eligieron tres motores relacionales muy usados, pero con diferencias claras en tipos de datos, manejo de JSON y reglas de integridad, para que la comparación tenga contenido técnico.

- **PostgreSQL 17.** Es el motor en el que ya se había trabajado el modelo en la Tarea 3, así que sirve como base de referencia. Es de código abierto, sigue de cerca el estándar SQL y tiene el tipo JSONB, pensado para documentos semiestructurados.
- **MySQL 8.4.** Es uno de los motores de código abierto más usados en aplicaciones web. Su tipo JSON nativo desde la versión 5.7 lo hace útil para comparar con JSONB. Su manejo de esquemas y de restricciones es distinto al de PostgreSQL.
- **SQL Server 2022.** Es el motor de Microsoft, muy presente en entornos empresariales, con herramientas como SQL Server Management Studio (SSMS), que aparece en el material de consulta de la unidad. No tiene tipo JSON nativo en esta versión, lo que obliga a resolver el JSON de otra forma y permite comparar tres estrategias distintas.

Motor	Versión probada	Entorno de prueba	Herramienta
PostgreSQL	17	Instalación local	pgAdmin 4 (Query Tool y ERD)
MySQL	8.4	En línea	db<>fiddle
SQL Server	2022 (Express, Linux)	En línea	db<>fiddle

3. Diccionario de datos genérico actualizado

Este diccionario describe la estructura lógica del modelo, sin tipos propios de ningún motor. Parte del modelo lógico normalizado de la Tarea 2 (tablas, atributos, claves primarias y foráneas) y se actualiza con una sola adición: el atributo **detalles_json** en la tabla *eventos*. Las tablas intermedias (usuarios_intereses, seguimientos, miembros_grupo y asistencias) resuelven las relaciones N:M.

3.1 Tablas y atributos

usuarios — Estudiantes registrados en la red social.

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_usuario	Identificador único del usuario.	Entero (autonumérico)	PK	Sí	Sí

nombre	Nombre completo del estudiante.	Texto (100)	—	Sí	No
correo	Correo electrónico de contacto.	Texto (150)	—	Sí	Sí
carrera	Programa académico que cursa.	Texto (80)	—	Sí	No

intereses_habilidades — Catálogo de intereses y habilidades (disciplinas, deportes, tecnologías).

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_interes	Identificador único del interés o habilidad.	Entero (autonumérico)	PK	Sí	Sí
nombre	Nombre del interés o habilidad.	Texto (60)	—	Sí	Sí

publicaciones — Mensajes publicados por los usuarios (preguntas, recursos, noticias, memes).

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_publicacion	Identificador único de la publicación.	Entero (autonumérico)	PK	Sí	Sí
contenido	Texto de la publicación.	Texto largo	—	Sí	No
fecha_publicacion	Fecha y hora en que se publicó.	Fecha y hora	—	Sí	No
id_usuario_autor	Usuario que publica.	Entero	FK → usuarios(id_usuario)	Sí	No

comentarios — Comentarios que los usuarios escriben sobre una publicación.

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_comentario	Identificador único del comentario.	Entero (autonumérico)	PK	Sí	Sí
contenido	Texto del comentario.	Texto (500)	—	Sí	No
id_usuario_autor	Usuario que escribe el comentario.	Entero	FK → usuarios(id_usuario)	Sí	No
id_publicacion	Publicación comentada.	Entero	FK → publicaciones(id_publicacion)	Sí	No

grupos — Grupos de estudio, equipos de hackatón o clubes de interés.

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_grupo	Identificador único del grupo.	Entero (autonumérico)	PK	Sí	Sí
nombre	Nombre del grupo.	Texto (100)	—	Sí	No
descripcion	Descripción o propósito del grupo.	Texto (300)	—	No	No
id_usuario_creador	Usuario que creó el grupo (opcional).	Entero	FK → usuarios(id_usuario)	No	No

eventos — Reuniones de estudio, talleres o actividades sociales programadas.

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_evento	Identificador único del evento.	Entero (autonumérico)	PK	Sí	Sí
nombre	Nombre del evento.	Texto (100)	—	Sí	No
fecha_evento	Fecha y hora del evento.	Fecha y hora	—	Sí	No
lugar	Lugar donde se realiza.	Texto (150)	—	Sí	No

id_usuario_organizador	Usuario que organiza el evento.	Entero	FK → usuarios(id_usuario)	No	No
id_grupo	Grupo que programa el evento (opcional).	Entero	FK → grupos(id_grupo)	No	No
detalles_json	Datos variables del evento en formato JSON: ubicación geográfica, aforo y lecturas de sensores de asistencia.	JSON (semiestructurado)	—	No	No

usuarios_intereses — Tabla intermedia de la relación N:M entre usuarios e intereses.

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_usuario	Usuario que posee el interés.	Entero	PK, FK → usuarios(id_usuario)	Sí	No
id_interes	Interés o habilidad del usuario.	Entero	PK, FK → intereses_ habilidades(id_inter es)	Sí	No

seguimientos — Relación reflexiva N:M: un usuario sigue a otro usuario.

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_seguidor	Usuario que sigue.	Entero	PK, FK → usuarios(id_usuario)	Sí	No
id_seguido	Usuario que es seguido.	Entero	PK, FK → usuarios(id_usuario)	Sí	No

miembros_grupo — Tabla intermedia de la relación N:M entre usuarios y grupos.

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_usuario	Usuario miembro.	Entero	PK, FK → usuarios(id_usuario)	Sí	No
id_grupo	Grupo al que pertenece.	Entero	PK, FK → grupos(id_grupo)	Sí	No

asistencias — Tabla intermedia de la relación N:M entre usuarios y eventos.

Atributo	Descripción	Tipo lógico	Clave	Oblig.	Único
id_usuario	Usuario que asiste.	Entero	PK, FK → usuarios(id_usuario)	Sí	No
id_evento	Evento al que asiste.	Entero	PK, FK → eventos(id_evento)	Sí	No

3.2 Relaciones del modelo

Relación	Tablas	Cardinalidad	Implementada en
PUBLICA	usuarios – publicaciones	1:N	publicaciones.id_usuario_autor
ESCRIBE	usuarios – comentarios	1:N	comentarios.id_usuario_autor
COMENTA_EN	publicaciones – comentarios	1:N	comentarios.id_publicacion
CREA	usuarios – grupos	1:N	grupos.id_usuario_creador
ORGANIZA	usuarios – eventos	1:N	eventos.id_usuario_organizador
PROGRAMA	grupos – eventos	1:N (opcional)	eventos.id_grupo
POSEE	usuarios – intereses_habilidades	N:M	usuarios_intereses (id_usuario, id_interes)
SIGUE	usuarios – usuarios	N:M reflexiva	seguimientos (id_seguidor, id_seguido)

PERTENECE_A	usuarios – grupos	N:M	miembros_grupo (id_usuario, id_grupo)
ASISTE_A	usuarios – eventos	N:M	asistencias (id_usuario, id_evento)

Sobre la normalización. El modelo se mantiene en tercera forma normal. Un campo JSON guarda varios datos dentro de una sola celda, así que en sentido estricto no cumple la primera forma normal. Se aceptó como decisión consciente porque esos datos cambian de un evento a otro y no sirven para relacionar tablas. Lo que sí relaciona (el organizador, el grupo y los asistentes) sigue en columnas y claves normales.

4. Datos semiestructurados: JSON para Big Data e IoT

4.1 Qué se agregó

Se agregó la columna **detalles_json** a la tabla *eventos*. Guarda en un documento JSON los datos que cambian según el tipo de evento: la ubicación con coordenadas geográficas, el aforo máximo y las lecturas de un sensor de asistencia (por ejemplo, un lector de códigos QR o un contador de personas en la puerta).

```
{
  "ubicacion": {"lat": 6.2776, "lng": -75.5686, "bloque": "Aula 204"},
  "aforo_max": 40,
  "sensor_asistencia": {"tipo": "QR", "lecturas": 32}
}
```

4.2 Justificación técnica y relación con Big Data e IoT

- **Estructura variable.** Un evento con sensor de asistencia guarda campos distintos a los de un taller sin sensor. Con columnas fijas habría que crear muchas columnas casi siempre vacías o cambiar la tabla cada vez que aparezca un dispositivo nuevo. Con JSON cada evento guarda solo lo que necesita.
- **Datos de dispositivos IoT.** Los sensores y lectores entregan sus mensajes en JSON y con campos que pueden cambiar según el fabricante. Guardarlos tal cual evita transformarlos antes de almacenarlos.
- **Coordenadas geográficas.** La latitud y la longitud se guardan como números dentro del documento. Quedan listas para analizarse con herramientas de datos geoespaciales, caso que tratan los materiales de consulta sobre motores híbridos SQL/NoSQL (Herrera-Ramírez et al., 2021; Hasan et al., 2019).
- **Enfoque híbrido.** Se mantiene el modelo relacional para lo que necesita integridad (usuarios, grupos, asistencias) y se usa JSON solo para lo flexible. Así se aprovechan las ventajas de ambos mundos sin migrar todo a una base NoSQL.

4.3 Tipo usado en cada motor

Aspecto	PostgreSQL	MySQL	SQL Server
Tipo de la columna	JSONB	JSON	NVARCHAR(MAX)
Validación del formato	Automática + CHECK con jsonb_typeof (debe ser objeto)	Automática por el tipo	CHECK (ISJSON(detalles_json) = 1)
Cómo se almacena	Binario	Binario	Texto
Función para leer un campo	-> y ->> (operadores)	JSON_EXTRACT, JSON_UNQUOTE	JSON_VALUE
Filtrar por contenido	Operador @> (contiene)	JSON_EXTRACT en el WHERE	JSON_VALUE en el WHERE
Índice sobre el JSON	Sí, GIN (creado en el script)	No directo (requiere columna generada)	No directo (requiere columna calculada)

4.4 Consultas de prueba

```
-- PostgreSQL
SELECT nombre, detalles_json->'ubicacion'->>'bloque' AS bloque,
       (detalles_json->>'aforo_max')::INT AS aforo_max
FROM eventos WHERE detalles_json @> '{"sensor_asistencia":{"tipo":"QR"}}';
```

```
-- MySQL
SELECT nombre, JSON_UNQUOTE(JSON_EXTRACT(detalles_json, '$.ubicacion.bloque')) AS bloque,
       JSON_EXTRACT(detalles_json, '$.aforo_max') AS aforo_max
FROM eventos WHERE JSON_UNQUOTE(JSON_EXTRACT(detalles_json, '$.sensor_asistencia.tipo')) = 'QR';

-- SQL Server
SELECT nombre, JSON_VALUE(detalles_json, '$.ubicacion.bloque') AS bloque,
       JSON_VALUE(detalles_json, '$.aforo_max') AS aforo_max
FROM red_social.eventos WHERE JSON_VALUE(detalles_json, '$.sensor_asistencia.tipo') = 'QR';
```

5. Implementación paso a paso

5.1 Cambios comunes a los tres motores

- Se conservaron las 10 tablas, los nombres de columnas, las claves y las restricciones del modelo lógico.
- Se agregó **detalles_json** a *eventos* y se creó, en cada motor, la forma más adecuada de validar y consultar el JSON.
- Cada script sigue el mismo orden: primero las tablas padre (usuarios, intereses_habilidades), luego las que dependen de ellas, después las tablas intermedias, los índices y por último datos de prueba con una consulta al JSON.
- Cada script está comentado y las restricciones usan los mismos prefijos: pk_, fk_, uq_, ck_ e ix_.

5.2 PostgreSQL 17 (pgAdmin 4)

- Conexión al servidor local en pgAdmin 4 y apertura de *Query Tool* sobre la base red_social_pascualina.
- Ejecución del script `ddl_postgresql.sql`: borra y vuelve a crear el esquema red_social, y usa `SET search_path` para crear allí las tablas.
- Creación de las 10 tablas con `GENERATED ALWAYS AS IDENTITY`, claves primarias, foráneas con ON DELETE / ON UPDATE, UNIQUE y CHECK. La columna **detalles_json** se define como JSONB.
- Creación de los índices sobre claves foráneas y del índice GIN sobre el JSON.
- Inserción de un usuario y de un evento con su documento JSON, y consulta con los operadores `->`, `->>` y `@>`.
- Generación del diagrama con la opción *ERD For Schema* de pgAdmin.

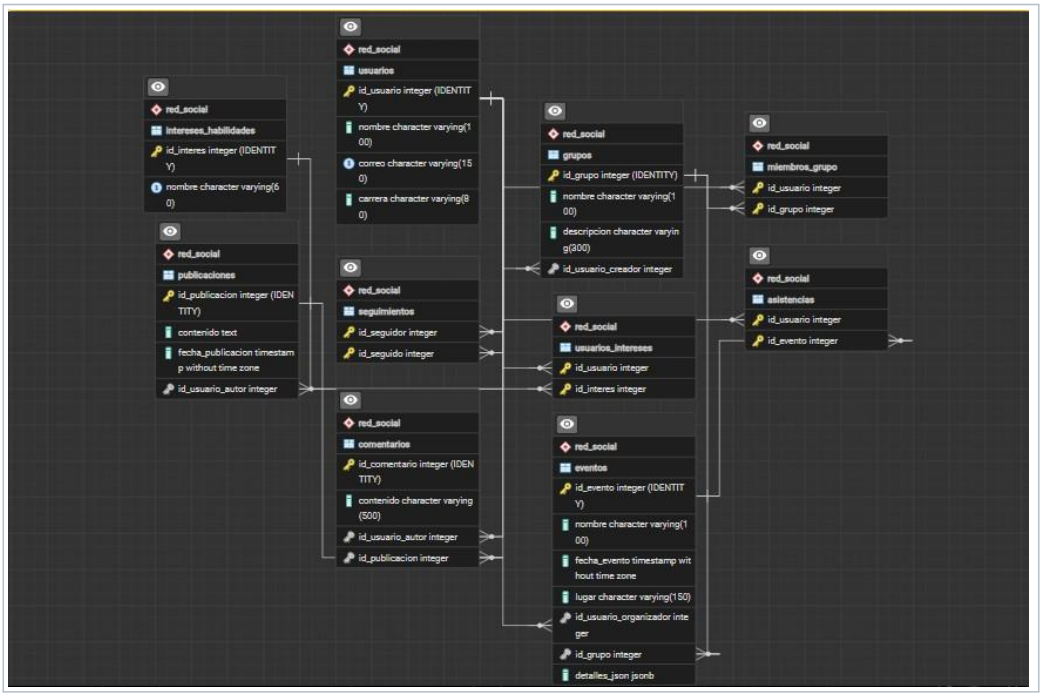


Figura 1. Diagrama del esquema red_social generado en pgAdmin. En la tabla eventos se ve la columna detalles_json de tipo jsonb.

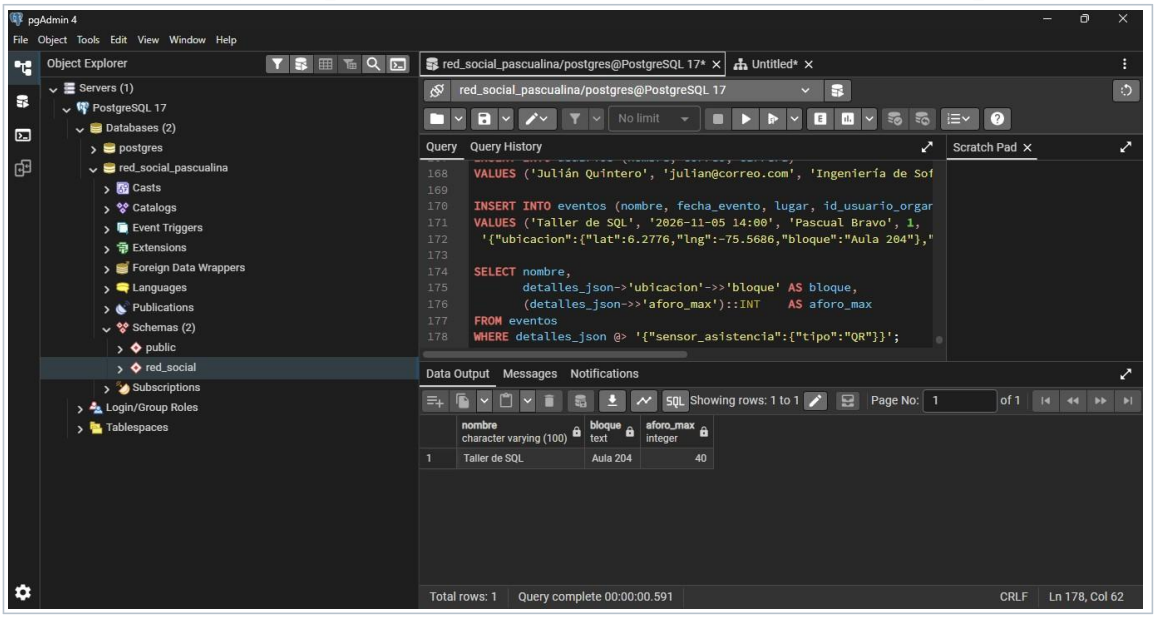


Figura 2. Consulta al campo JSONB en pgAdmin: devuelve el evento «Taller de SQL», el bloque «Aula 204» y el aforo 40.

5.3 MySQL 8.4 (db<>fiddle)

- Selección de MySQL 8.4 en db<>fiddle. Como la página ya entrega una base de datos, se usó la versión del script sin `DROP DATABASE`, `CREATE DATABASE` ni `USE`.
- Pegado del script de creación en el cuadro de esquema y de la consulta al JSON en el cuadro de consulta.
- Cambios de tipos respecto a PostgreSQL: `AUTO_INCREMENT`, `DATETIME` y `JSON`.
- Cambio en *seguimientos*: las claves foráneas usan `ON UPDATE RESTRICT` (ver sección 7).
- Ejecución: los cinco `CREATE INDEX` finales responden sin advertencias y la consulta devuelve el mismo resultado que en PostgreSQL.

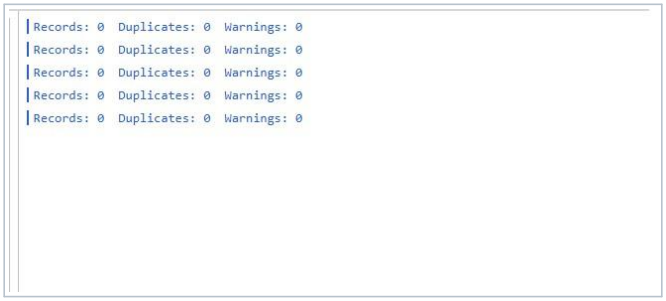


Figura 3. Resultado de la ejecución en MySQL 8.4: los cinco CREATE INDEX terminan con Records: 0, Duplicates: 0, Warnings: 0 (sin errores).

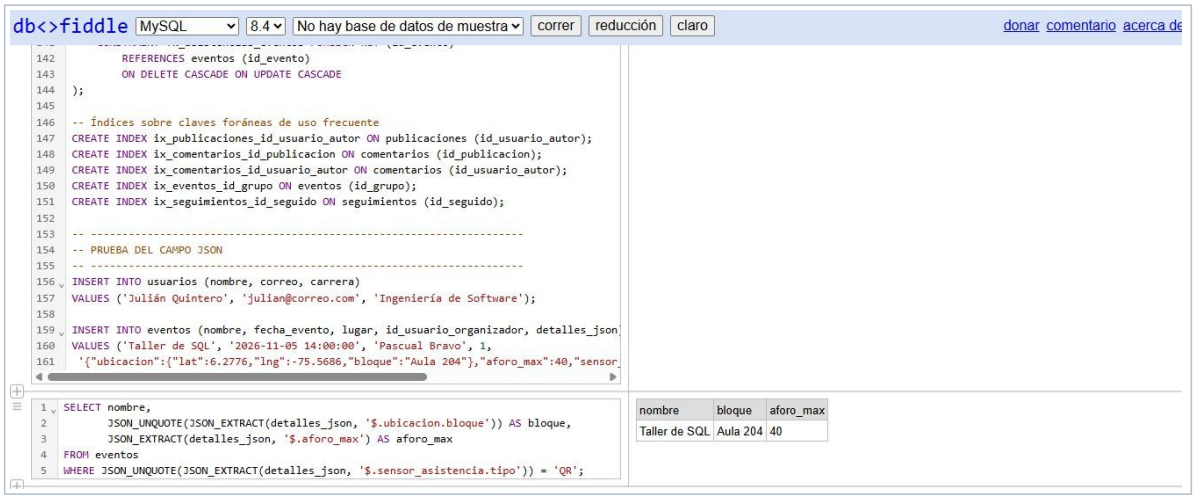


Figura 4. Final del script de MySQL (índices e inserts de prueba) y consulta al campo JSON con JSON_EXTRACT.

5.4 SQL Server 2022 (db<>fiddle)

- Selección de SQL Server 2022 en db<>fiddle. Se quitaron `CREATE DATABASE`, `USE` y los `GO`, porque la herramienta no los maneja.
- El esquema se crea con `EXEC('CREATE SCHEMA red_social')`, ya que esa instrucción debe ir sola en su lote.
- Cambios de tipos: `IDENTITY(1,1)`, `NVARCHAR`, `DATETIME2(0)` y `NVARCHAR(MAX)` con `CHECK (ISJSON(...) = 1)` para el JSON.
- Cuatro claves foráneas se definen con `NO ACTION` por la regla de rutas de cascada (ver sección 7).
- Ejecución: el script corre completo y el mensaje «2 rows affected» corresponde a los dos INSERT de prueba. La consulta con `JSON_VALUE` devuelve el mismo resultado que en los otros motores.

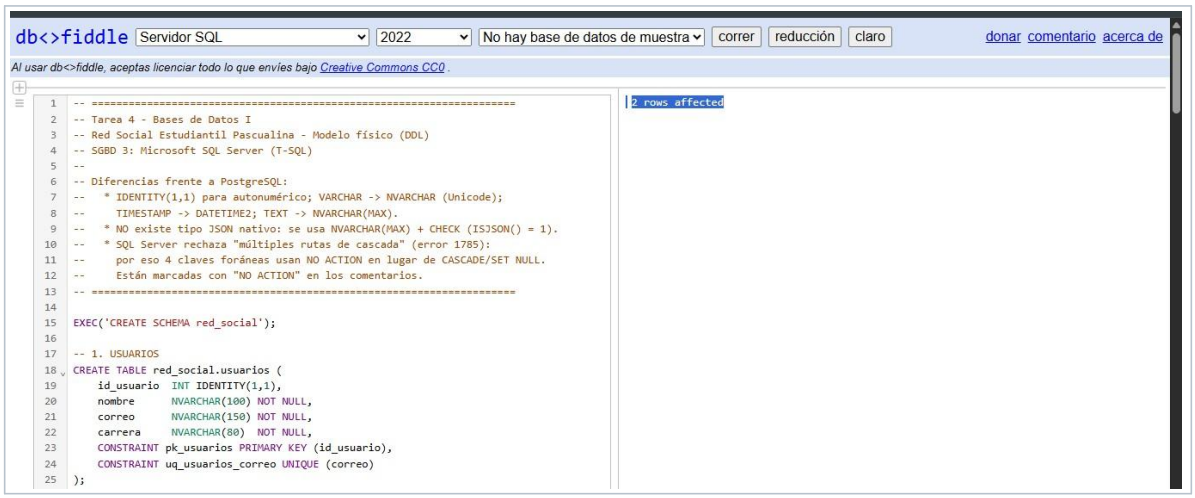


Figura 5. Script de SQL Server pegado en db<>fiddle y ejecutado; a la derecha aparece «2 rows affected», sin mensajes de error.

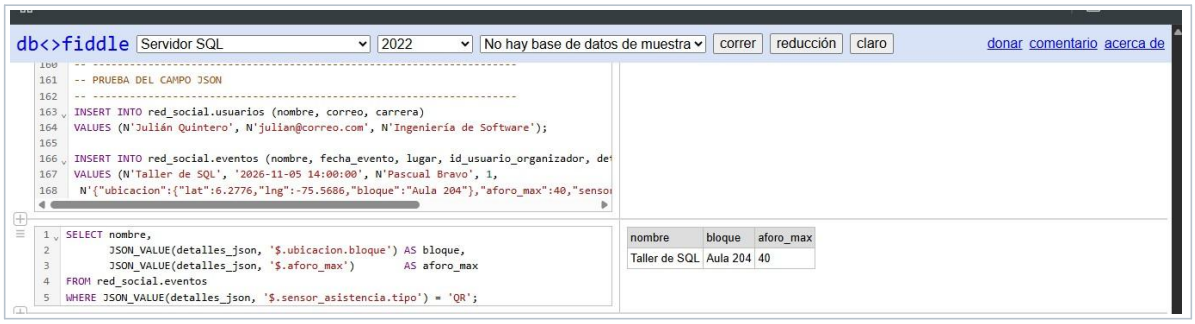


Figura 6. Final del script (inserts de prueba) y consulta con JSON_VALUE en SQL Server 2022. Se recortó la publicidad de la página.

6. Diccionarios de datos físicos

Cada diccionario lista, por tabla, el tipo real de cada columna, su tamaño, si acepta nulos, las claves y las restricciones propias del motor. Los mismos diccionarios se entregan por separado en la carpeta *diccionarios* del repositorio.

6.1 Diccionario físico de PostgreSQL 17

Ubicación: Base red_social_pascualina, esquema red_social.

Consideraciones particulares del motor

- Se usa **IDENTITY** (estándar SQL) en lugar de SERIAL para los autonuméricos.
- Se separa el modelo en un **esquema propio** (red_social) mediante `SET search_path`.
- **JSONB** guarda el documento en formato binario: permite índice GIN y operadores como `->`, `->>` y `@>`. El CHECK con `jsonb_typeof` obliga a que sea un objeto y no un valor suelto.
- Permite combinar CHECK con acciones referenciales (ON UPDATE CASCADE) y varias rutas de cascada sin restricciones adicionales.

usuarios

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INTEGER	4 bytes	No	PK	GENERATED ALWAYS AS IDENTITY; PK pk_usuarios
nombre	VARCHAR(100)	100 caracteres	No	—	—
correo	VARCHAR(150)	150 caracteres	No	—	UNIQUE uq_usuarios_correo
carrera	VARCHAR(80)	80 caracteres	No	—	—

intereses_habilidades

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_interes	INTEGER	4 bytes	No	PK	GENERATED ALWAYS AS IDENTITY; PK pk_intereses_habilidades
nombre	VARCHAR(60)	60 caracteres	No	—	UNIQUE uq_intereses_habilidades_nombre

publicaciones

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_publicacion	INTEGER	4 bytes	No	PK	GENERATED ALWAYS AS IDENTITY; PK pk_publicaciones
contenido	TEXT	Hasta 1 GB	No	—	—
fecha_publicacion	TIMESTAMP	8 bytes	No	—	DEFAULT CURRENT_TIMESTAMP

id_usuario_autor	INTEGER	4 bytes	No	FK	FK fk_publicaciones_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE

comentarios

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_comentario	INTEGER	4 bytes	No	PK	GENERATED ALWAYS AS IDENTITY; PK pk_comentarios
contenido	VARCHAR(500)	500 caracteres	No	—	—
id_usuario_autor	INTEGER	4 bytes	No	FK	FK fk_comentarios_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_publicacion	INTEGER	4 bytes	No	FK	FK fk_comentarios_publicaciones → publicaciones(id_publicacion). ON DELETE CASCADE, ON UPDATE CASCADE

grupos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_grupo	INTEGER	4 bytes	No	PK	GENERATED ALWAYS AS IDENTITY; PK pk_grupos
nombre	VARCHAR(100)	100 caracteres	No	—	—
descripcion	VARCHAR(300)	300 caracteres	Sí	—	—
id_usuario_creador	INTEGER	4 bytes	Sí	FK	FK fk_grupos_usuarios → usuarios(id_usuario). ON DELETE SET NULL, ON UPDATE CASCADE

eventos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_evento	INTEGER	4 bytes	No	PK	GENERATED ALWAYS AS IDENTITY; PK pk_eventos
nombre	VARCHAR(100)	100 caracteres	No	—	—
fecha_evento	TIMESTAMP	8 bytes	No	—	—
lugar	VARCHAR(150)	150 caracteres	No	—	—
id_usuario_organizador	INTEGER	4 bytes	Sí	FK	FK fk_eventos_usuarios → usuarios(id_usuario). ON DELETE SET NULL, ON UPDATE CASCADE
id_grupo	INTEGER	4 bytes	Sí	FK	FK fk_eventos_grupos → grupos(id_grupo). ON DELETE CASCADE, ON UPDATE CASCADE
detalles_json	JSONB	Variable (binario)	Sí	—	CHECK ck_eventos_detalles_objeto: debe ser un objeto JSON. Índice GIN ix_eventos_detalles_json

usuarios_intereses

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INTEGER	4 bytes	No	PK, FK	PK pk_usuarios_intereses (compuesta); FK fk_usuarios_intereses_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE

id_interes	INTEGER	4 bytes	No	PK, FK	PK pk_usuarios_intereses (compuesta); FK fk_usuarios_intereses_intereses → intereses_habilidades(id_interes). ON DELETE CASCADE, ON UPDATE CASCADE

seguimientos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_seguidor	INTEGER	4 bytes	No	PK, FK	PK pk_seguimientos (compuesta); FK fk_seguimientos_seguidor → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_seguido	INTEGER	4 bytes	No	PK, FK	PK pk_seguimientos (compuesta); FK fk_seguimientos_seguido → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE

CHECK ck_seguimientos_no_autoseguir: id_seguidor <> id_seguido (un usuario no puede seguirse a sí mismo).

miembros_grupo

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INTEGER	4 bytes	No	PK, FK	PK pk_miembros_grupo (compuesta); FK fk_miembros_grupo_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_grupo	INTEGER	4 bytes	No	PK, FK	PK pk_miembros_grupo (compuesta); FK fk_miembros_grupo_grupos → grupos(id_grupo). ON DELETE CASCADE, ON UPDATE CASCADE

asistencias

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INTEGER	4 bytes	No	PK, FK	PK pk_asistencias (compuesta); FK fk_asistencias_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_evento	INTEGER	4 bytes	No	PK, FK	PK pk_asistencias (compuesta); FK fk_asistencias_eventos → eventos(id_evento). ON DELETE CASCADE, ON UPDATE CASCADE

Índices

Índice	Tabla (columna)	Motivo
ix_publicaciones_id_usuario_autor	publicaciones (id_usuario_autor)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_comentarios_id_publicacion	comentarios (id_publicacion)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_comentarios_id_usuario_autor	comentarios (id_usuario_autor)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_eventos_id_grupo	eventos (id_grupo)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_seguimientos_id_seguido	seguimientos (id_seguido)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_eventos_detalles_json	eventos USING GIN (detalles_json)	Consultas dentro del JSON (operador @>)

6.2 Diccionario físico de MySQL 8.4

Ubicación: Base de datos red_social_pascualina (en MySQL esquema y base de datos son lo mismo).

Consideraciones particulares del motor

- En MySQL **esquema y base de datos son lo mismo**: se crea la base red_social_pascualina y las tablas quedan dentro de ella.
- **AUTO_INCREMENT** reemplaza a IDENTITY; **DATETIME** reemplaza a TIMESTAMP (TIMESTAMP en MySQL tiene rango limitado hasta 2038).
- El tipo **JSON** es nativo, valida el formato al insertar y se consulta con `JSON_EXTRACT` y `JSON_UNQUOTE`. No se puede indexar directamente; para indexar un campo habría que crear una columna generada.
- MySQL no permite que una columna usada en un **CHECK** tenga **ON UPDATE CASCADE** (error 3823). Por eso en *seguimientos* las dos claves foráneas usan ON UPDATE RESTRICT. **usuarios**

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INT	4 bytes	No	PK	AUTO_INCREMENT; PK pk_usuarios
nombre	VARCHAR(100)	100 caracteres	No	—	—
correo	VARCHAR(150)	150 caracteres	No	—	UNIQUE uq_usuarios_correo
carrera	VARCHAR(80)	80 caracteres	No	—	—

intereses_habilidades

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_interes	INT	4 bytes	No	PK	AUTO_INCREMENT; PK pk_intereses_habilidades
nombre	VARCHAR(60)	60 caracteres	No	—	UNIQUE uq_intereses_habilidades_nombre

publicaciones

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_publicacion	INT	4 bytes	No	PK	AUTO_INCREMENT; PK pk_publicaciones
contenido	TEXT	Hasta 64 KB	No	—	—
fecha_publicacion	DATETIME	5 bytes	No	—	DEFAULT CURRENT_TIMESTAMP
id_usuario_autor	INT	4 bytes	No	FK	FK fk_publicaciones_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE

comentarios

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_comentario	INT	4 bytes	No	PK	AUTO_INCREMENT; PK pk_comentarios
contenido	VARCHAR(500)	500 caracteres	No	—	—
id_usuario_autor	INT	4 bytes	No	FK	FK fk_comentarios_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_publicacion	INT	4 bytes	No	FK	FK fk_comentarios_publicaciones → publicaciones(id_publicacion). ON DELETE CASCADE, ON UPDATE CASCADE

grupos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones

id_grupo	INT	4 bytes	No	PK	AUTO_INCREMENT; PK pk_grupos
nombre	VARCHAR(100)	100 caracteres	No	—	—
descripcion	VARCHAR(300)	300 caracteres	Sí	—	—
id_usuario_creador	INT	4 bytes	Sí	FK	FK fk_grupos_usuarios → usuarios(id_usuario). ON DELETE SET NULL, ON UPDATE CASCADE

eventos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_evento	INT	4 bytes	No	PK	AUTO_INCREMENT; PK pk_eventos
nombre	VARCHAR(100)	100 caracteres	No	—	—
fecha_evento	DATETIME	5 bytes	No	—	—
lugar	VARCHAR(150)	150 caracteres	No	—	—
id_usuario_organizador	INT	4 bytes	Sí	FK	FK fk_eventos_usuarios → usuarios(id_usuario). ON DELETE SET NULL, ON UPDATE CASCADE
id_grupo	INT	4 bytes	Sí	FK	FK fk_eventos_grupos → grupos(id_grupo). ON DELETE CASCADE, ON UPDATE CASCADE
detalles_json	JSON	Variable (binario)	Sí	—	El tipo JSON valida el formato automáticamente

usuarios_intereses

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INT	4 bytes	No	PK, FK	PK pk_usuarios_intereses (compuesta); FK fk_usuarios_intereses_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_interes	INT	4 bytes	No	PK, FK	PK pk_usuarios_intereses (compuesta); FK fk_usuarios_intereses_intereses → intereses_habilidades(id_interes). ON DELETE CASCADE, ON UPDATE CASCADE

seguimientos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_seguidor	INT	4 bytes	No	PK, FK	PK pk_seguimientos (compuesta); FK fk_seguimientos_seguidor → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE RESTRICT
id_seguido	INT	4 bytes	No	PK, FK	PK pk_seguimientos (compuesta); FK fk_seguimientos_seguido → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE RESTRICT

CHECK ck_seguimientos_no_autoseguir: id_seguidor <> id_seguido (un usuario no puede seguirse a sí mismo).

miembros_grupo

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones

id_usuario	INT	4 bytes	No	PK, FK	PK pk_miembros_grupo (compuesta); FK fk_miembros_grupo_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_grupo	INT	4 bytes	No	PK, FK	PK pk_miembros_grupo (compuesta); FK fk_miembros_grupo_grupos → grupos(id_grupo). ON DELETE CASCADE, ON UPDATE CASCADE

asistencias

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INT	4 bytes	No	PK, FK	PK pk_asistencias (compuesta); FK fk_asistencias_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_evento	INT	4 bytes	No	PK, FK	PK pk_asistencias (compuesta); FK fk_asistencias_eventos → eventos(id_evento). ON DELETE CASCADE, ON UPDATE CASCADE

Índices

Índice	Tabla (columna)	Motivo
ix_publicaciones_id_usuario_autor	publicaciones (id_usuario_autor)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_comentarios_id_publicacion	comentarios (id_publicacion)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_comentarios_id_usuario_autor	comentarios (id_usuario_autor)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_eventos_id_grupo	eventos (id_grupo)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_seguimientos_id_seguido	seguimientos (id_seguido)	Acelera JOIN y borrados en cascada sobre la clave foránea

6.3 Diccionario físico de SQL Server 2022

Ubicación: Base red_social_pascualina, esquema red_social.

Consideraciones particulares del motor

- Se usa el **esquema red_social** dentro de la base. `CREATE SCHEMA` debe ejecutarse sola en su lote (por eso se usó `EXEC` en db<>fiddle).
- **NVARCHAR** (Unicode) reemplaza a VARCHAR: guarda 2 bytes por carácter. **DATETIME2(0)** reemplaza a TIMESTAMP (que en SQL Server no es fecha, es un contador de versión de fila). **TEXT** está obsoleto, por eso se usa NVARCHAR(MAX).
- En la versión 2022 **no existe un tipo JSON nativo**: el JSON se guarda como NVARCHAR(MAX) y el CHECK con `ISJSON(...) = 1` garantiza que sea válido. Se consulta con `JSON_VALUE`.
- SQL Server rechaza varias **rutas de cascada** hacia una misma tabla (error 1785). Cuatro claves foráneas pasan a **NO ACTION**: fk_comentarios_usuarios, fk_grupos_usuarios, fk_eventos_usuarios y fk_seguimientos_seguido. **usuarios**

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INT	4 bytes	No	PK	IDENTITY(1,1); PK pk_usuarios
nombre	NVARCHAR(100)	100 caracteres (200 bytes)	No	—	—

correo	NVARCHAR(150)	150 caracteres (300 bytes)	No	—	UNIQUE uq_usuarios_correo
carrera	NVARCHAR(80)	80 caracteres (160 bytes)	No	—	—

intereses_habilidades

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_interes	INT	4 bytes	No	PK	IDENTITY(1,1); PK pk_intereses_habilidades
nombre	NVARCHAR(60)	60 caracteres (120 bytes)	No	—	UNIQUE uq_intereses_habilidades_nombre

publicaciones

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_publicacion	INT	4 bytes	No	PK	IDENTITY(1,1); PK pk_publicaciones
contenido	NVARCHAR(MAX)	Hasta 2 GB	No	—	—
fecha_publicacion	DATETIME2(0)	6 bytes	No	—	DEFAULT SYSDATETIME()
id_usuario_autor	INT	4 bytes	No	FK	FK fk_publicaciones_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE

comentarios

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_comentario	INT	4 bytes	No	PK	IDENTITY(1,1); PK pk_comentarios
contenido	NVARCHAR(500)	500 caracteres (1000 bytes)	No	—	—
id_usuario_autor	INT	4 bytes	No	FK	FK fk_comentarios_usuarios → usuarios(id_usuario). ON DELETE NO ACTION, ON UPDATE NO ACTION
id_publicacion	INT	4 bytes	No	FK	FK fk_comentarios_publicaciones → publicaciones(id_publicacion). ON DELETE CASCADE, ON UPDATE CASCADE

grupos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_grupo	INT	4 bytes	No	PK	IDENTITY(1,1); PK pk_grupos
nombre	NVARCHAR(100)	100 caracteres (200 bytes)	No	—	—
descripcion	NVARCHAR(300)	300 caracteres (600 bytes)	Sí	—	—
id_usuario_creador	INT	4 bytes	Sí	FK	FK fk_grupos_usuarios → usuarios(id_usuario). ON DELETE NO ACTION, ON UPDATE NO ACTION

eventos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_evento	INT	4 bytes	No	PK	IDENTITY(1,1); PK pk_eventos
nombre	NVARCHAR(100)	100 caracteres (200 bytes)	No	—	—

fecha_evento	DATETIME2(0)	6 bytes	No	—	—
lugar	NVARCHAR(150)	150 caracteres (300 bytes)	No	—	—
id_usuario_organizador	INT	4 bytes	Sí	FK	FK fk_eventos_usuarios → usuarios(id_usuario). ON DELETE NO ACTION, ON UPDATE NO ACTION
id_grupo	INT	4 bytes	Sí	FK	FK fk_eventos_grupos → grupos(id_grupo). ON DELETE CASCADE, ON UPDATE CASCADE
detalles_json	NVARCHAR(MAX)	Hasta 2 GB (texto)	Sí	—	CHECK ck_eventos_detalles_json: ISJSON(detalles_json) = 1

usuarios_intereses

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INT	4 bytes	No	PK, FK	PK pk_usuarios_intereses (compuesta); FK fk_usuarios_intereses_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_interes	INT	4 bytes	No	PK, FK	PK pk_usuarios_intereses (compuesta); FK fk_usuarios_intereses_intereses → intereses_habilidades(id_interes). ON DELETE CASCADE, ON UPDATE CASCADE

seguimientos

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_seguidor	INT	4 bytes	No	PK, FK	PK pk_seguimientos (compuesta); FK fk_seguimientos_seguidor → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_seguido	INT	4 bytes	No	PK, FK	PK pk_seguimientos (compuesta); FK fk_seguimientos_seguido → usuarios(id_usuario). ON DELETE NO ACTION, ON UPDATE NO ACTION

CHECK ck_seguimientos_no_autoseguir: id_seguidor <> id_seguido (un usuario no puede seguirse a sí mismo).

miembros_grupo

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INT	4 bytes	No	PK, FK	PK pk_miembros_grupo (compuesta); FK fk_miembros_grupo_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE
id_grupo	INT	4 bytes	No	PK, FK	PK pk_miembros_grupo (compuesta); FK fk_miembros_grupo_grupos → grupos(id_grupo). ON DELETE CASCADE, ON UPDATE CASCADE

asistencias

Columna	Tipo	Tamaño	Nulo	Clave	Restricciones y observaciones
id_usuario	INT	4 bytes	No	PK, FK	PK pk_asistencias (compuesta); FK fk_asistencias_usuarios → usuarios(id_usuario). ON DELETE CASCADE, ON UPDATE CASCADE

id_evento	INT	4 bytes	No	PK, FK	PK pk_asistencias (compuesta); FK fk_asistencias_eventos → eventos(id_evento). ON DELETE CASCADE, ON UPDATE CASCADE

Índices

Índice	Tabla (columna)	Motivo
ix_publicaciones_id_usuario_autor	publicaciones (id_usuario_autor)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_comentarios_id_publicacion	comentarios (id_publicacion)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_comentarios_id_usuario_autor	comentarios (id_usuario_autor)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_eventos_id_grupo	eventos (id_grupo)	Acelera JOIN y borrados en cascada sobre la clave foránea
ix_seguimientos_id_seguido	seguimientos (id_seguido)	Acelera JOIN y borrados en cascada sobre la clave foránea

7. Dificultades encontradas

7.1 MySQL: CHECK incompatible con ON UPDATE CASCADE (error 3823)

En la tabla *seguimientos* hay un CHECK que impide que un usuario se siga a sí mismo (`id_seguidor <> id_seguido`). En PostgreSQL las dos claves foráneas usan ON UPDATE CASCADE sin problema. En MySQL esa combinación no es válida: una columna usada en un CHECK no puede tener una acción referencial como CASCADE. Para comprobarlo se cambió temporalmente RESTRICT por CASCADE y MySQL rechazó la creación de la tabla.

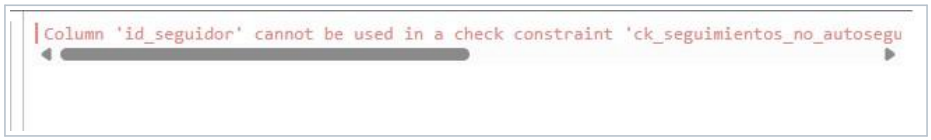


Figura 7. Error de MySQL al usar ON UPDATE CASCADE en una columna que participa en el CHECK ck_seguimientos_no_autoseguir (mensaje cortado por el ancho del cuadro de resultados).

Solución: usar `ON UPDATE RESTRICT` en las dos claves foráneas. No se pierde funcionalidad real, porque id_usuario es autonumérico y no se modifica. El script final, con esa corrección, es el que ejecutó sin errores.

```
-- 8. SEGUIMIENTOS
-- ON UPDATE RESTRICT: MySQL prohíbe CHECK sobre columnas con ON UPDATE CASCADE.
-- Como id_usuario es AUTO_INCREMENT y no cambia, no se pierde funcionalidad.
CREATE TABLE seguimientos (
    id_seguidor  INT NOT NULL,
    id_seguido   INT NOT NULL,
    CONSTRAINT pk_seguimientos PRIMARY KEY (id_seguidor, id_seguido),
    CONSTRAINT fk_seguimientos_seguidor FOREIGN KEY (id_seguidor)
        REFERENCES usuarios (id_usuario)
        ON DELETE CASCADE ON UPDATE RESTRICT,
    CONSTRAINT fk_seguimientos_seguido FOREIGN KEY (id_seguido)
        REFERENCES usuarios (id_usuario)
        ON DELETE CASCADE ON UPDATE RESTRICT,
    CONSTRAINT ck_seguimientos_no_autoseguir CHECK (id_seguidor <> id_seguido)
);
```

Figura 8. Definición final de seguimientos en MySQL, con ON UPDATE RESTRICT y el comentario que explica el cambio.

7.2 SQL Server: varias rutas de cascada (error 1785)

SQL Server no permite que una misma tabla se alcance por dos caminos de borrado o actualización en cascada. En este modelo pasaba, por ejemplo, con *comentarios*: al borrar un usuario se llegaba a sus comentarios directamente y también a través de sus publicaciones. Algo parecido ocurría con *seguimientos*, que tiene dos claves hacia *usuarios*, y con las tablas que dependen de *grupos* y *eventos*. Esta diferencia se identificó al revisar el modelo antes de ejecutar el script, por eso no hay captura del error.

Solución: definir `NO ACTION` en cuatro claves foráneas: fk_comentarios_usuarios, fk_grupos_usuarios, fk_eventos_usuarios y fk_seguimientos_seguido. La consecuencia es que no se puede borrar un usuario que tenga esos registros asociados sin eliminarlos antes. En PostgreSQL esos casos se resolvían solos con CASCADE o SET NULL.

7.3 Tres formas distintas de manejar JSON

El mismo dato se guarda y se consulta distinto en cada motor (JSONB con operadores, JSON con funciones y NVARCHAR con ISJSON). Las consultas no se pueden copiar de un motor a otro; hubo que reescribir cada una y comprobar que devolvieran el mismo resultado.

7.4 Probar sin instalar los motores

MySQL y SQL Server se probaron en db<>fiddle para cumplir la fecha de entrega. La herramienta ya trae una base de datos y no maneja `GO`, así que hubo que preparar una versión de cada script para ella. Además, al no tener los motores locales, no se pudieron generar diagramas propios de MySQL y SQL Server como el de pgAdmin.

8. Ventajas y desventajas observadas

8.1 Por motor

Motor	Ventajas observadas	Desventajas observadas
PostgreSQL	JSONB con índice GIN y operadores de contenido. Acepta CHECK junto con ON UPDATE CASCADE y varias rutas de cascada. IDENTITY estándar. pgAdmin genera el diagrama del esquema.	Hay que decidir entre JSON y JSONB. Es menos común que SQL Server en entornos empresariales con Windows.
MySQL	Tipo JSON nativo que valida el formato solo. Sintaxis sencilla (AUTO_INCREMENT). Es muy usado en aplicaciones web.	No permite CHECK con acciones referenciales (error 3823). El JSON no se indexa directamente. Esquema y base de datos son lo mismo.
SQL Server	Esquemas separados dentro de la base. Funciones JSON (ISJSON, JSON_VALUE) fáciles de usar. Integración con SSMS.	Sin tipo JSON nativo en la versión 2022. Restricción de rutas de cascada (error 1785). NVARCHAR ocupa el doble de bytes. CREATE SCHEMA debe ir sola en su lote.

8.2 Comparación técnica

Aspecto	PostgreSQL	MySQL	SQL Server
Autonumérico	GENERATED ALWAYS AS IDENTITY	AUTO_INCREMENT	IDENTITY(1,1)
Fecha y hora	TIMESTAMP	DATETIME	DATETIME2(0)
Texto largo	TEXT (hasta 1 GB)	TEXT (hasta 64 KB)	NVARCHAR(MAX) (hasta 2 GB)
Texto con límite	VARCHAR(n)	VARCHAR(n)	NVARCHAR(n)
Agrupación de objetos	Esquema dentro de la base	La base de datos es el esquema	Esquema dentro de la base
JSON	JSONB	JSON	NVARCHAR(MAX) + ISJSON
Índice sobre JSON	GIN	Columna generada	Columna calculada
CHECK + ON UPDATE CASCADE	Permitido	Error 3823	Permitido

Varias rutas de cascada	Permitido	Permitido	Error 1785

9. Conclusiones generales y propuestas de mejora

9.1 Conclusiones generales del proceso

- El modelo lógico normalizado se pudo llevar a los tres motores sin cambiar su estructura: las 10 tablas, las claves y las relaciones se mantuvieron. Lo que cambió fueron los tipos de datos, la sintaxis y algunas reglas de integridad.
- Los tres motores comparten el estándar SQL básico, pero no sirve copiar un script de uno a otro: cada motor tiene reglas propias, como se vio con el error 3823 en MySQL y la regla de rutas de cascada en SQL Server. Por eso hay que probar el script en cada uno.
- Incluir JSON dentro de un modelo relacional es una buena solución para datos cambiantes como los de sensores y coordenadas, siempre que se use solo para lo flexible y no para reemplazar las claves y relaciones. El soporte para consultarlo es parecido en los tres motores, pero PostgreSQL ofrece más herramientas (JSONB e índice GIN).
- Documentar con diccionarios genérico y físicos ayuda a ver qué cambia de un motor a otro y deja un registro claro de cada decisión.

9.2 Propuestas de mejora

- **Indexar campos del JSON** en MySQL (columna generada) y SQL Server (columna calculada) para que las consultas por aforo o tipo de sensor no tengan que leer todos los documentos.
- **Separar las lecturas de sensores** en una tabla propia (por ejemplo, lecturas_sensores con fecha, evento y valor) si el volumen crece mucho, en lugar de guardarlas todas dentro del JSON del evento.
- **Restaurar el comportamiento de SET NULL y CASCADE en SQL Server** mediante disparadores (triggers) o procedimientos almacenados, para no depender de NO ACTION.
- **Probar con más datos** (varios usuarios, grupos y eventos) y ejecutar borrados y actualizaciones para verificar el efecto real de cada regla ON DELETE y ON UPDATE.
- **Instalar los tres motores** de forma local para generar diagramas propios y medir tiempos de respuesta de las consultas al JSON.
- **Trabajo en equipo:** repartir un motor por integrante y comparar propuestas en el foro antes de fijar la versión final.

10. Conclusiones individuales

Lo que más me costó fue pasar el modelo a otros motores sin perder lo que ya tenía definido. Pensé que bastaba con cambiar los tipos de datos, pero las reglas de integridad también cambian. En MySQL, la tabla de seguimientos no me dejó usar ON UPDATE CASCADE junto con el CHECK que evita que un usuario se siga a sí mismo, y tuve que pasar a RESTRICT. En SQL Server tuve que dejar cuatro claves foráneas en NO ACTION por la regla de las rutas de cascada, y eso significa que ya no se borra un usuario con la misma facilidad que en PostgreSQL.

Lo que más me llamó la atención fue lo distinto que se maneja el JSON: en PostgreSQL es un tipo propio con operadores y un índice GIN, en MySQL es un tipo nativo que se lee con funciones como JSON_EXTRACT, y en SQL Server es texto con un CHECK de ISJSON que se lee con JSON_VALUE. Es el mismo dato guardado de tres formas. También entendí mejor para qué sirve en un caso de IoT: un evento con sensor y otro sin sensor guardan información distinta, y con JSON no hay que crear columnas vacías para todos.

Aprendí que probar el script en cada motor es obligatorio. El error 3823 de MySQL solo lo entendí al verlo en pantalla, y también aprendí a documentar las diferencias en el diccionario físico para no olvidarlas. Me quedó claro que normalizar y diseñar bien el modelo desde el principio facilita mucho llevarlo a otro motor.

Si lo repitiera, probaría los tres scripts desde el inicio y no solo el de PostgreSQL, instalaría los motores en mi computador para ver los diagramas de cada uno y llenaría las tablas con más datos de prueba para revisar cómo se comportan los borrados. Como trabajé solo, también me hizo falta comparar mi propuesta con la de otros compañeros para encontrar errores que yo no veía.

11. Organización del repositorio Git y video

Los entregables están organizados según la estructura pedida en la tarea:

```
Tarea4/
├── diccionarios/
│   ├── diccionario-generico/        (diccionario de datos genérico actualizado)
│   ├── diccionario-sgbd-1/          (PostgreSQL)
│   ├── diccionario-sgbd-2/          (MySQL)
│   └── diccionario-sgbd-3/          (SQL Server)
├── ddl/
│   ├── scripts-ddl-sgbd-1/          (ddl_postgresql.sql)
│   ├── scripts-ddl-sgbd-2/          (ddl_mysql.sql y versión para db<>fiddle)
│   └── scripts-ddl-sgbd-3/          (ddl_sqlserver.sql y versión para db<>fiddle)
├── informe/                         (este informe en PDF y carpeta capturas/)
└── video/                           (enlace al video en YouTube)
```

Video de sustentación: [pegar aquí el enlace de YouTube]. Tiene una duración de 5 a 10 minutos y se presentó de forma individual, con las conclusiones personales de la sección 10.

Referencias

Fadhel, S. A. y Jameel, E. A. (2022). A comparison between nosql and rdbms: storage and retrieval. *Minar International Journal of Applied Sciences and Technology, 4*(3), 172-184. https://doi.org/10.47832/2717-8234.12.18

Hasan, M., Panidi, E. y Badenko, V. (2019). Comparative evaluation of nosql and relational databases performance while analyzing semi-structured geospatial data. International Scientific Conference GEOBALCANICA. https://doi.org/10.18509/gbp.2019.64

Herrera-Ramírez, J., Treviño-Villalobos, M. y Víquez-Acuña, L. (2021). Un motor de almacenamiento híbrido para datos geoespaciales utilizando paradigmas NoSQL y SQL. *Revista Tecnología en Marcha, 34*(1), 40-54. https://doi.org/10.18845/tm.v34i1.4822

Microsoft Ignite. (2025, 12 de junio). ¿Qué es SQL Server Management Studio (SSMS)? https://learn.microsoft.com/es-es/ssms/sql-server-management-studio-ssms